REC HOSTEL – SMART DIGITAL GATE-PASS MANAGEMENT SYSTEM

PHASE I PROJECT REPORT

Submitted by

Maahesh S – 241701025

Pradeep Kumar R – 241701042

in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING
COMPUTER SCIENCE AND DESIGN





RAJALAKSHMI ENGINEERING COLLEGE
ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE
DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

BONAFIDE CERTIFICATE

Certified that this report entitled "REC HOSTEL – SMART DIGITAL GATE-PASS MANAGEMENT SYSTEM" is the bonafide work of Maahesh S – 241701025 and Pradeep Kumar R – 241701042, submitted as part of the Phase I project requirements in the Department of Computer Science and Design.

The report documents the proposed problem, requirements, workflow, architecture, database model, validation approach, and planned testing. Statements about implementation and evaluation are limited to the project information available at the time of preparation.

SIGNATURE OF HOD	**SIGNATURE OF SUPERVISOR**
Name: _____	Name: _____
Head of the Department	Designation: _____
Department of Computer Science and Design	

Submitted for the Phase I Project Viva-Voce examination held on: _____.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ABSTRACT

The manual hostel gate-pass process may rely on paper forms, handwritten approvals, physical registers, and separate communication between students, parents, wardens, and security personnel. These methods can make approval status difficult to trace and historical entry or exit records time-consuming to retrieve. REC Hostel – Smart Digital Gate-Pass Management System is proposed as a role-based web application that centralises gate-pass requests, approvals, QR-based gate verification, and notifications.

The system supports five roles: Student, Parent, Warden, Watchman, and Admin. A student submits an outing reason, exit date and time, expected return date and time, and optional destination, emergency contact, or supporting attachment. Identity fields such as name, college email, phone number, department, hostel, register number, and room number are drawn from the student profile and remain read-only in the request form.

Linked parents receive a request and complete the configured approval requirement. The request then reaches authorised wardens assigned to the student's hostel; any one assigned warden can make the decision, with the first valid decision retained. A temporary QR code is generated only after required approvals. Watchman scans record OUT and IN events, notify linked parents, and support overdue-return and late-entry alerts. The report describes the project's design, requirements, data model, security controls, test plan, expected benefits, and future work. Planned functionality is not presented as proven deployment performance.

Keywords: *digital gate pass, hostel, QR code, role-based access, parent approval, warden approval, audit trail.*

ACKNOWLEDGEMENT

We express our sincere gratitude to the management of Rajalakshmi Engineering College for providing the academic environment in which this project can be planned and documented. We thank the Principal and the Department of Computer Science and Design for their encouragement and support.

We convey our thanks to the Head of the Department, project coordinator, faculty members, and project supervisor for their guidance and suggestions. Their academic feedback helps us organise the project requirements and develop a systematic approach to analysis and design.

We acknowledge the needs of students, parents, wardens, and hostel security personnel, whose everyday responsibilities motivate a clearer and more traceable gate-pass process. The proposed design considers each of these roles and defines permissions and actions appropriate to their responsibilities.

Finally, we thank our families and friends for their encouragement during the preparation of this Phase I report.

Maahesh S – 241701025
Pradeep Kumar R – 241701042

TABLE OF CONTENTS

Chapter 1 Introduction **1**
1.1 General 1
1.2 Background 1
1.3 Problem Statement 1
1.4 Motivation 1
1.5 Objectives 1
1.6 Scope 2
1.7 Existing System 2
1.8 Limitations 2
1.9 Proposed System 3
1.10 Expected Benefits 3
Chapter 2 Literature Review **4**
2.1 Introduction 4
2.2 Review Approach 4
2.3 Comparative Analysis 6
2.4 Design Gap Addressed 6
Chapter 3 Theoretical Background **8**
3.1 General 8
3.2 Design Principles 8
3.3 Workflow 8
3.4 Role Boundaries 8
3.5 Request States 9
3.6 Architecture Overview 9
3.7 Context DFD 10
3.8 Level 1 DFD 11
3.9 Detailed Flow 11
3.10 Use Cases 11
3.11 Activity Flow 12
3.12 Sequence 12
3.13 Core Classes 13
Chapter 4 Methodology **14**
4.1 General 14
4.2 Methodology 14
4.3 Requirement Analysis 14
4.4 Functional Requirements 14
4.5 Non-Functional Requirements 15
4.6 Technology Selection 16
4.7 Student Module 16
4.8 Parent Module 17
4.9 Warden Module 17
4.10 Watchman Module 18
4.11 Admin Module 18
4.12 Form Design 18
4.13 Approval Rules 19
4.14 Database Design 19
4.15 Relationships 20
4.16 Technical Feasibility 21
4.17 Testing Strategy 21

4.18 Test Case Set A....22
4.19 Test Case Set B....22
4.20 Test Case Set C....23
4.21 Expected Results....23
4.22 Challenges....24
Chapter 5 Conclusion....25
5.1 Conclusion....25
5.2 Phase I Status....25
5.3 Limitations....25
5.4 Phase II Objectives....25
5.5 Future Enhancements....26
5.6 Work Schedule....26
References....27
Appendix A CO and SDG Relevance....28
A.1 Purpose....28
Appendix B Program Outcomes....28
B.1 Suggested Outcome Evidence....28
Appendix C Program Specific Outcomes....29
C.1 PSO Evidence....29
Appendix D CO–PO–PSO Mapping Template....29
D.1 Mapping Instructions....29

LIST OF TABLES

Table 1.1 Limitations of the existing process....2
Table 2.1 Request states....5
Table 2.2 Role-wise permitted actions....5
Table 2.3 Manual versus proposed process....7
Table 3.1 Role-permission matrix....9
Table 3.2 Gate-pass state transitions....9
Table 3.3 Actors and primary use cases....12
Table 3.4 Conceptual classes and entities....13
Table 4.1 Functional requirements....15
Table 4.2 Non-functional requirements....15
Table 4.3 Proposed technology stack....16
Table 4.4 Request form fields....19
Table 4.5 Approval and cancellation rules....19
Table 4.6 Database entities....20
Table 4.7 Relationships and integrity....21
Table 4.8 Feasibility assessment....21
Table 4.9 Test cases: authentication and requests....22
Table 4.10 Test cases: approval and QR....23
Table 4.11 Test cases: return and notifications....23
Table 5.1 Risks and mitigation....25
Table 5.2 Phase II work schedule....26
Table A.1 CO and SDG relevance record....28
Table B.1 Suggested outcome evidence....28
Table C.1 PSO evidence template....29
Table D.1 CO–PO–PSO mapping template....29

LIST OF FIGURES

Figure 3.1 Overall system architecture.....10
Figure 3.2 Context-level data flow diagram.....10
Figure 3.3 Level 1 data flow diagram.....11
Figure 3.4 Gate-pass activity flow.....12
Figure 3.5 Approval and scan sequence.....13

Note: Diagrams are presented in text-based form so that they remain editable; they can be converted to formal UML/DFD artwork for final submission while retaining the same logic.

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
DBMS	Database Management System
DFD	Data Flow Diagram
FR	Functional Requirement
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
IN	Recorded hostel entry event
NFR	Non-Functional Requirement
OUT	Recorded hostel exit event
QR	Quick Response
RBAC	Role-Based Access Control
SDG	Sustainable Development Goal
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1
INTRODUCTION

1.1 General

A hostel gate-pass process controls and records student movement outside the hostel or campus. The process requires a reliable link between the student requesting permission, the parent or guardian confirming the request, the warden responsible for approval, and the watchman who verifies the pass at the gate. A digital workflow can connect these responsibilities without removing the institution's approval rules.

1.2 Background

Paper forms and physical registers are straightforward to introduce, but information can be distributed across separate documents and communication channels. A central application can store requests, decisions, QR pass status, and entry or exit events in one structured database. It also gives each role a tailored view of the information required for its task.

1.3 Problem Statement

The project addresses the absence of a single traceable workflow for recording gate-pass requests, parent approvals, warden decisions, QR verification, and actual gate movement. The intended solution is a secure web application that shows the current request state and retains an audit history.

1.4 Motivation

The motivation is to improve transparency, reduce repeated status enquiries, minimise dependence on paper records, and make return-time exceptions visible to authorised parents. The design emphasises clear approval responsibility and accurate recording of actual entry and exit rather than merely producing a digital form.

- Keep request information in one record.
- Make the required next approver visible.
- Prevent QR generation before approval is complete.
- Record gate events with a timestamp and staff account.
- Notify linked parents about important state changes.

1.5 Objectives

The system should manage the full pass lifecycle, from request submission through approvals, exit, return, and final history. It should also support profile management, parent linkage, warden-to-hostel assignment, cancellation, and controlled administrative reporting.

1.5.1 Success Criteria

Success is evaluated through functional checks rather than assumed benefits. A successful prototype should demonstrate that each role can perform only its authorised actions, approvals follow the configured sequence, a QR is unavailable before approval, scan states are recorded correctly, and late-return notifications are generated under the specified conditions.

- All five roles can authenticate and access their permitted dashboard.
- Required parent approvals gate the warden stage.

- Any one assigned warden can make the valid warden decision.
- Duplicate or out-of-order scans do not corrupt the request state.
- Cancellation invalidates a pass when the rules permit cancellation.
- Audit records allow authorised staff to reconstruct important events.

1.6 Scope

The current scope includes the five role dashboards, profile and master-data management, gate-pass requests, parent and warden approvals, temporary QR generation, watchman scans, notifications, cancellation, and audit history.

1.6.1 Included

Student details are drawn from the profile and are read-only in the request form. The form includes reason, exit date and time, expected return date and time, and optional destination, emergency contact, and attachment. Overnight requests are supported when the dates and times are valid.

1.6.2 Excluded

GPS tracking, biometric verification, IVR calls, SOS workflows, and AI prediction are not part of the agreed version. External messaging requires configuration and is not guaranteed by a user-interface prototype alone.

- A live deployment requires hosting, database, backups, and access control configuration.
- Policies such as whether one parent or all required parents must approve should be configurable.
- Actual usability and performance must be measured during testing.

1.7 Existing System

The baseline process may use paper forms, handwritten signatures, parent calls or messages, and a physical register maintained at the gate. The student presents the approved pass, and the security person records departure or return using the available local procedure.

1.8 Limitations

Paper can be misplaced, handwritten entries can be difficult to search, and updates may not reach all stakeholders at the same time. A manual process may also lack a standard way to connect an approval to the exact entry and exit events.

Table 1.1 Limitations of the existing process

Existing approach	Risk or limitation
Paper form	Loss, damage, and limited searchability
Separate phone or message confirmation	Approval status may be unclear or hard to audit
Manual warden signature	Request state may not be visible remotely
Physical register	Historical movement records can take time to retrieve
Unlinked parent records	Notifications may reach an incomplete set of contacts

1.9 Proposed System

The proposed web application stores each gate-pass request, its approvers, its QR token status, and its scan events. It routes the request to linked parents first, then to the authorised warden group for the student's hostel. After approval, the watchman uses the QR code to record departure and return.

1.10 Expected Benefits

The system is expected to make request status more visible, reduce repeated manual coordination, improve the consistency of movement records, and provide parents with event-based notifications. These are expected outcomes to be verified by testing, not measured claims at the design stage.

- Centralised status and history.
- Defined role permissions and approval sequence.
- Time-stamped OUT and IN records.
- Overdue and late-entry alerts.
- Controlled cancellation and QR invalidation.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

This chapter reviews the established concepts used in the design: web-based information systems, workflow state management, role-based access control, QR codes, relational databases, and event notifications. These concepts provide a technical basis for implementing a traceable gate-pass process.

2.2 Review Approach

The project is a practical system design rather than a claim of a new algorithm. The review therefore compares the roles of common technology patterns and explains how each pattern addresses a requirement in the proposed application. Final institutional references should be checked against the exact versions of the technologies selected for implementation.

2.2.1 Web-Based Information Systems

A web application can be opened in a browser without requiring each stakeholder to install a separate desktop program. A responsive layout adapts forms, status cards, and approval controls to different screen sizes.

2.2.2 Relevance to REC Hostel

Students may apply from a desktop or phone, parents may review a request on a phone, and staff may use an office computer or a gate device. The interface should provide the same request state and permission rules across devices while adapting layout to screen width.

- Use clear navigation by role.
- Show status, timestamps, and next action.
- Provide form validation close to the relevant input.
- Ensure controls remain usable on smaller screens.
- Do not rely on colour alone to communicate approval state.

2.2.3 Workflow Management

A workflow defines the sequence of activities and the conditions for moving between them. Explicit states make it possible to distinguish a draft request from one awaiting parent approval, one awaiting a warden, one approved for exit, one already exited, and one returned.

2.2.4 State-Based Design

State transitions should be validated on the server. A request cannot be treated as approved merely because the browser displays an approval button or because a client submits an arbitrary status value. The backend must confirm the user, role, assignment, current state, and applicable policy before accepting a transition.

Table 2.1 Request states

State	Meaning
PENDING_PARENT	Waiting for required parent approval
PENDING_WARDEN	Parent stage complete; awaiting warden decision
APPROVED	All required approvals complete; pass can be issued
REJECTED	A required approval rejected the request
CANCELLED	Request cancelled under policy
OUT	Valid pass has recorded an exit
RETURNED	IN event recorded
OVERDUE	Return deadline passed without an IN event

2.2.5 RBAC Concept

Role-based access control groups permissions by responsibility. A role does not automatically grant access to every record: record-level conditions are also required. For example, a parent must be linked to the student, and a warden must be assigned to the relevant hostel.

2.2.6 Enforcement

The interface should hide actions that a role cannot use, but this is only a usability measure. The server or database must independently enforce the same permissions. Every decision should be associated with the authenticated user and a timestamp.

Table 2.2 Role-wise permitted actions

Role	Typical permitted actions
Student	Create and view own requests; cancel where allowed; view QR
Parent	View and decide requests for linked student(s)
Warden	Decide requests for assigned hostel(s)
Watchman	Validate QR and record OUT/IN events
Admin	Manage accounts, profiles, hostels, assignments, and reports

2.2.7 QR Codes

A QR code is a machine-readable representation of data. In this application, it should carry an opaque pass token or a signed reference rather than personal details. The backend resolves the token and checks the pass record before allowing a gate action.

2.2.8 Validation Requirements

A QR scan is not proof of approval by itself. The server checks that the token exists, the pass is approved, the pass has not been cancelled or expired, the current time falls within the configured exit window, and the requested state transition is valid. A scan result should clearly say ALLOW or DON’T ALLOW with a short reason.

- Use sufficiently unpredictable tokens.
- Do not encode parent phone numbers or other private data in the QR.
- Invalidate the token after cancellation or other policy-defined invalidation.
- Record scan user, time, result, and action.

- Prevent duplicate OUT and duplicate IN state transitions.

2.2.9 Date and Time

The system must validate that the expected return is later than the permitted exit time and that required date-time values are present. Overnight requests must be supported by comparing full date-time values rather than comparing clock times alone. The system should display the time zone used for policy decisions.

2.2.10 Notification Processing

Notifications should be generated from accepted events, such as a request submission, approval, rejection, exit, return, overdue deadline, late entry, or cancellation. In-app notifications should be stored so that a user can review them later. External email or push delivery depends on configured providers.

- Store event timestamps consistently.
- Schedule overdue checks using the expected return time.
- Notify parents when the return deadline passes without an IN event.
- When the student returns late, send a separate notice with actual entry time and lateness duration.

2.2.11 Relational Data Model

A relational database represents entities in tables and connects them using primary and foreign keys. It suits this application because users, student profiles, parents, hostels, warden assignments, requests, approvals, QR tokens, scans, notifications, and audit events have defined relationships.

2.2.12 Integrity

Constraints should prevent orphaned records and invalid references. Transactions are important when a warden decision or gate scan changes more than one related record. For example, the accepted approval and request status should be updated atomically so that two concurrent decisions cannot both become authoritative.

2.3 Comparative Analysis

The proposed system differs from a paper process by using one shared request record, explicit approval states, temporary QR validation, and time-stamped gate events. It differs from a basic online form by modelling the complete lifecycle and applying role and assignment checks.

2.4 Design Gap Addressed

A request-only application may collect details but leave parent approval, warden assignment, gate verification, return recording, and overdue handling disconnected. The project addresses this workflow gap through a single stateful process. This is a design rationale and should not be represented as a claim that all existing systems lack these features.

Table 2.3 Manual versus proposed process

Capability	Manual process	Proposed system
Request status	May require staff enquiry	Stored status visible to authorised users
Approval order	Depends on local practice	Defined parent then warden sequence
Gate validation	Paper/signature check	Server-validated temporary QR
Movement history	Physical register	Time-stamped OUT/IN events
Late return	Manual follow-up	Deadline monitoring and notifications
Audit history	Scattered paper trail	Linked approval, cancellation, and scan records

CHAPTER 3
THEORETICAL BACKGROUND

3.1 General

The design combines an authenticated web interface, a server-side business layer, persistent storage, and a QR scanning interface. The user interface collects input and displays state; the business layer applies policy; the database preserves records and history.

3.2 Design Principles

The main principles are least privilege, explicit state transitions, data minimisation, validation at trust boundaries, auditability, and separation of user interface from business rules. These principles reduce the risk that a display-only restriction is mistaken for a security control.

3.3 Workflow

The student opens the request form. Profile fields are populated from the stored student profile. The student enters the outing reason, exit date and time, expected return date and time, and any optional fields. The system validates the values and assigns a unique request ID.

3.3.1 Parent Stage

The system notifies all linked parents. The configured policy determines whether one parent approval is sufficient or all designated required parents must approve. If a required parent rejects, the request is rejected and the warden stage is not opened.

3.3.2 Warden Stage

After the parent requirement is satisfied, the request becomes available to the authorised wardens assigned to that hostel. Any one such warden can approve or reject. A first-response-wins transaction prevents conflicting decisions.

- All required approvals complete → generate QR.
- Valid QR at exit → record OUT and notify parents.
- Valid QR at return → record IN and notify parents.
- Deadline passes with no IN → overdue notice.
- Late return scan → accept genuine return and send late-entry notice.

3.4 Role Boundaries

Each role should receive only the data required to perform its tasks. A parent must not search or view unrelated students. A warden’s authority is limited to assigned hostels. A watchman needs a clear scan result and the minimum student/pass details necessary for gate verification.

Table 3.1 Role-permission matrix

Function	Student	Parent	Warden	Watchman	Admin
Submit request	Yes	No	No	No	Support only
Approve parent stage	No	Linked student	No	No	Policy/config
Approve warden stage	No	No	Assigned hostel	No	Audit only
View student QR	Own pass	No	No	Scan only	No routine need
Record gate scan	No	No	No	Yes	Audit view
Manage profiles	No	No	No	No	Yes
View audit reports	Own history	Linked history	Assigned hostel	Own scans	Yes

3.5 Request States

The application should store a clear state value and validate each transition. Status labels in the interface should be readable, while the database uses consistent enumerated values. The implementation should distinguish approval from physical movement: an approved pass is not yet an OUT event, and an OUT event is not yet a return.

3.5.1 Concurrency

If two assigned wardens act at nearly the same time, the backend should accept only the first valid decision. The second attempt should receive a message that the request has already been decided. Similar protection is required for duplicate gate scans.

Table 3.2 Gate-pass state transitions

Current state	Event	Next state
DRAFT	Student submits valid request	PENDING_PARENT
PENDING_PARENT	Required parent approvals complete	PENDING_WARDEN
PENDING_PARENT	Required parent rejects	REJECTED
PENDING_WARDEN	Assigned warden approves	APPROVED
PENDING_WARDEN	Assigned warden rejects	REJECTED
APPROVED	Valid exit scan	OUT
OUT	Valid return scan	RETURNED
Pending/approved	Allowed cancellation	CANCELLED
OUT	Return deadline passes	OVERDUE (alert flag/event)

3.6 Architecture Overview

The logical architecture consists of the presentation layer, application or API layer, authentication and authorisation services, database layer, QR generation and verification

component, notification service, and scheduled deadline checker. The deployment may combine some of these services in one application, but the logical responsibilities should remain separate.

3.6.1 Presentation Layer

The browser interface contains role-specific dashboards, forms, approval queues, request history, and a scan screen. It should not be trusted to decide whether a pass is valid.

3.6.2 Business and Data Layers

The backend checks permissions, validates input, performs state transitions, issues or invalidates QR tokens, writes audit events, and triggers notifications. The database stores durable records and supports transaction-safe updates.

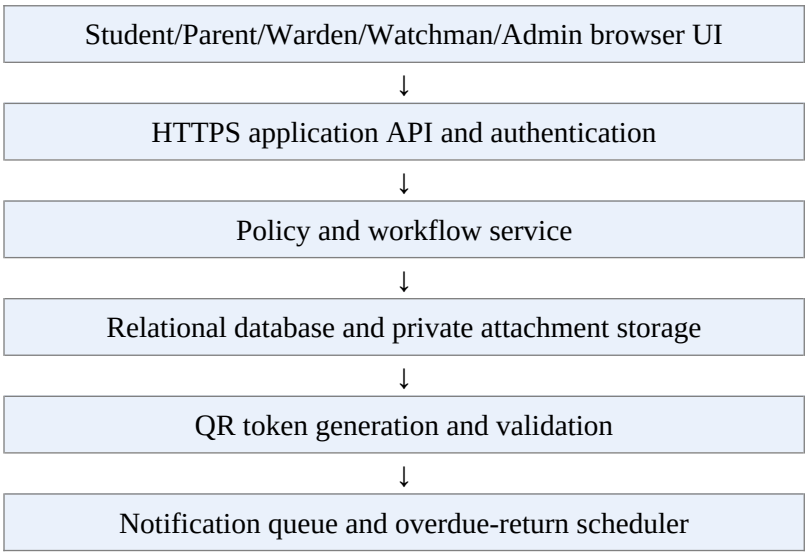


Figure 3.1 Overall system architecture

3.7 Context DFD

At the context level, the REC Hostel application is treated as one process interacting with five external actors. Each actor sends inputs and receives outputs according to role permissions. The database is internal to the application boundary.

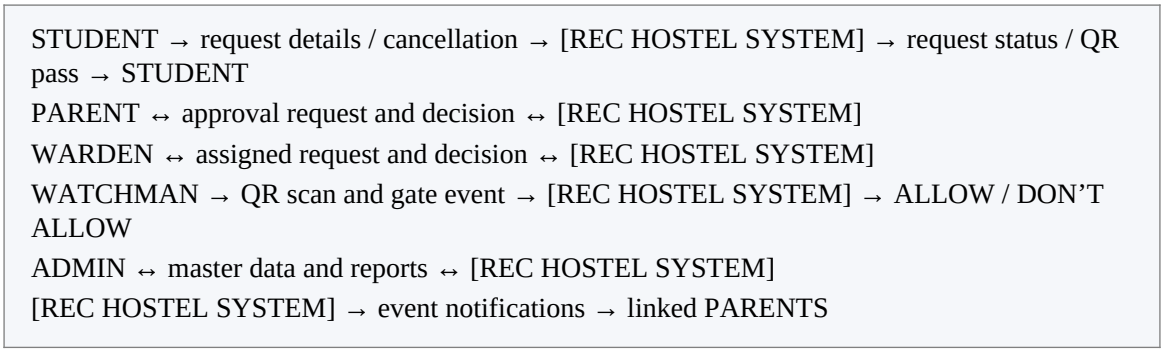


Figure 3.2 Context-level data flow diagram

3.8 Level 1 DFD

The system can be decomposed into profile management, request management, parent approval, warden approval, QR management, gate scan recording, notification management, and reporting. Each process reads or writes only the records necessary for its task.

P1 Manage users and profiles ↔ D1 User / Profile / Hostel Data
P2 Submit and validate request ↔ D2 Gate-Pass Requests
P3 Collect parent approvals ↔ D3 Approval Records
P4 Collect warden decision ↔ D3 Approval Records
P5 Issue or invalidate QR ↔ D4 QR Token Records
P6 Validate scans and record OUT/IN ↔ D5 Gate Scan Events
P7 Create and deliver notifications ↔ D6 Notification Records
P8 Generate authorised reports ← D1–D6

Figure 3.3 Level 1 data flow diagram

Every write operation must enforce authentication, role permissions, and valid state transitions.

3.9 Detailed Flow

When a student submits a request, the backend validates required fields, dates, student profile, and any policy-specific attachment requirement. A valid request is saved with a unique identifier and a pending-parent status. Parent notifications are created for linked contacts.

3.9.1 Approval Flow

The parent policy is evaluated. When all required approvals are complete, the request moves to the warden queue for the student's hostel. A warden decision is accepted only if the account is active, the warden is assigned to that hostel, and the request is still pending. The system stores the decision and time.

3.9.2 Gate Scan Flow

At exit, the scan service checks the token, approval status, cancellation status, expiry, time window, and current movement state. A valid scan records OUT. At return, a valid token associated with an OUT event records IN. The system does not refuse a genuine return solely because the student is late.

- Invalid token → DON'T ALLOW; log failed scan if policy permits.
- Cancelled or not-yet-approved pass → DON'T ALLOW.
- Duplicate OUT or IN → reject duplicate transition.
- Return after deadline → record IN and send late-entry alert.

3.10 Use Cases

The Student actor submits and views requests, sees the issued QR, and cancels where policy permits. The Parent actor views requests for linked students and submits an approval or rejection. The Warden actor reviews requests for assigned hostels and records one decision. The Watchman actor scans a QR and receives an allow/deny result. The Admin actor

maintains users, student profiles, parent links, hostels, warden assignments, and authorised reports.

Table 3.3 Actors and primary use cases

Actor	Primary use cases
Student	Login; submit request; view status; view QR; cancel permitted request; view history
Parent	Login; view linked request; approve/reject; view movement notifications
Warden	Login; view assigned queue; approve/reject; view hostel request history
Watchman	Login; scan QR; receive scan decision; view own scan history
Admin	Manage users; manage hostels; link parents; assign wardens; configure policy; audit

3.11 Activity Flow

The main activity starts when a student opens a new request. The system loads the student profile and validates the form. Invalid input returns to the form with clear errors; valid input creates the request and sends parent notifications.

3.11.1 Decision Branches

A rejection at the required parent stage ends the approval path as REJECTED. If parent requirements are complete, the warden queue opens. Warden rejection ends the approval path; approval issues the QR. The student then presents the QR at the gate, where the scan service validates the next allowed movement.

START → Load profile → Enter request details → Validate form
Invalid? YES → Show field errors → Return to form
Invalid? NO → Save request → Notify linked parents
Parent policy complete? NO → Wait for required decisions
Parent rejected? YES → REJECTED → Notify student
Parent complete? YES → Warden queue → Warden decision
Warden rejected? YES → REJECTED → Notify student/parents as configured
Warden approved? YES → Generate QR → Exit scan → OUT → Return scan → IN → END

Figure 3.4 Gate-pass activity flow

3.12 Sequence

The sequence begins with a student request submitted to the application. The application saves the request and creates notifications. Parent decisions are sent to the backend, which checks the parent-student relationship and policy. The warden decision is then checked against the hostel assignment and request state.

3.12.1 Scan Sequence

The watchman’s device submits a token to the scan endpoint. The backend validates the token and current state, then records the accepted scan in a transaction. The response is returned to the scan screen and parent notifications are queued. A failed validation returns a reason without exposing unnecessary private information.

Student → API: Submit request
API → Database: Validate and save request
API → Parent: Notify request
Parent → API: Approve / reject
API → Database: Validate relationship and store decision
API → Assigned warden: Notify pending decision
Warden → API: Approve / reject
API → Database: Store decision; issue QR if approved
Watchman → Scan API: Submit QR token
Scan API → Database: Validate state; record OUT or IN
Notification service → Parent: Send movement/late-return notice

Figure 3.5 Approval and scan sequence

3.13 Core Classes

The conceptual model separates identity, hostel organisation, request workflow, approvals, QR credentials, gate events, notifications, and audit history. This separation helps maintain data consistency and supports reporting without storing the same fact in multiple unrelated places.

Table 3.4 Conceptual classes and entities

Class / entity	Purpose
UserAccount	Login identity, account status, and role assignment
StudentProfile	Student name, register number, department, contact, hostel, room
ParentLink	Relationship between a parent account and a student
Hostel	Hostel master record
WardenAssignment	Maps an authorised warden to a hostel
GatePassRequest	Reason, time window, destination, status, and request ID
ApprovalRecord	Approver, decision, timestamp, and decision stage
QRPass	Opaque token reference, validity, and revocation status
GateScanEvent	OUT/IN action, timestamp, operator, and result
Notification	Recipient, event, message, read state, and timestamp
AuditEvent	Actor, action, target, time, and relevant change metadata

CHAPTER 4
METHODOLOGY

4.1 General

The project follows a structured software development process: requirements identification, analysis, design, implementation planning, validation, and evaluation. Each stage produces an artefact that can be reviewed before moving to the next stage.

4.2 Methodology

The workflow begins by defining actors and business rules. These are translated into functional requirements, role permissions, state transitions, database entities, and test cases. The application is then developed incrementally, with the end-to-end pass lifecycle tested after each relevant module is integrated.

- Requirement collection and clarification.
- System and data modelling.
- Interface and database design.
- Incremental implementation.
- Unit, integration, security, and usability testing.
- Documentation and review.

4.3 Requirement Analysis

Requirement analysis defines what the system must do and the conditions under which it must do it. The most important rule is that the gate-pass lifecycle is a controlled sequence, not an independent collection of forms.

4.3.1 Stakeholders

Students initiate requests; parents provide required confirmation; wardens make the institutional decision; watchmen enforce the gate process; and administrators maintain accounts and configuration. Each stakeholder requires a clear, limited view of the request and its status.

- Identify mandatory versus optional form fields.
- Confirm the institution’s parent approval policy.
- Define which wardens are assigned to each hostel.
- Define exit time windows and late-return handling.
- Agree cancellation conditions.
- Define notification channels and retention rules.

4.4 Functional Requirements

Functional requirements describe observable actions the system must support. They should be tested using valid, invalid, authorised, and unauthorised scenarios. The following list defines the minimum functional baseline for the proposed version.

Table 4.1 Functional requirements

ID	Requirement
FR-01	Authenticate users and route them to a role-appropriate dashboard.
FR-02	Allow Admin to create and manage accounts, profiles, hostels, and assignments.
FR-03	Link multiple parent accounts to a student where required.
FR-04	Auto-fill student profile fields and keep them read-only in the request form.
FR-05	Validate reason, exit date/time, and expected return date/time.
FR-06	Support optional destination, emergency contact, and attachment.
FR-07	Notify linked parents and collect configured approval requirements.
FR-08	Route parent-approved requests to assigned wardens.
FR-09	Accept one valid warden decision using first-response-wins.
FR-10	Generate a temporary QR only after required approvals.
FR-11	Validate QR and record OUT/IN scans.
FR-12	Send movement, overdue, and late-entry notifications.
FR-13	Support cancellation under policy and invalidate the QR.
FR-14	Maintain audit history and authorised reports.

4.5 Non-Functional Requirements

Non-functional requirements define quality expectations. They should be made measurable during implementation wherever possible, rather than described only with broad words such as fast, secure, or user-friendly.

Table 4.2 Non-functional requirements

Category	Requirement
Security	Server-side role and record-level access control; secure sessions; private file access.
Integrity	Transaction-safe approvals and scans; referential integrity; unique request IDs.
Usability	Clear labels, readable status, useful validation messages, responsive layouts.
Performance	Common dashboard and scan operations should respond within an agreed target under expected load.
Availability	Deployment should have a documented recovery and backup approach.
Auditability	Record important decisions, cancellations, and scan events with actor and timestamp.
Maintainability	Separate UI, workflow logic, persistence, and notification responsibilities.
Privacy	Collect only needed data and limit access to student and parent information.
Compatibility	Support current mainstream mobile and desktop browsers used by stakeholders.

4.6 Technology Selection

The final stack should align with the current application environment and the team's ability to maintain it. If the prototype already has a connected database or framework, the existing configuration should be reused rather than replaced without a clear reason.

4.6.1 Suggested Stack

A suitable option is a React and TypeScript front end, a server-side API, and a relational database such as PostgreSQL. Authentication and private file storage may be provided by a managed backend service when configured correctly. QR generation and scanning can be implemented using compatible libraries, while notifications can use in-app records and configured external services.

Table 4.3 Proposed technology stack

Layer	Suggested technology / approach
Frontend	React with TypeScript and responsive CSS/Tailwind
Backend	Server-side API or trusted backend service
Database	PostgreSQL or the project's existing relational backend
Authentication	Managed authentication or secure server-side sessions
QR	Opaque random token; server-side validation
Attachments	Private object storage with type and size validation
Testing	Unit, integration, role-permission, and end-to-end tests
Version control	Git repository with environment secrets excluded

4.6.2 Rationale

React supports reusable role dashboards and forms. TypeScript can detect certain data-shape mistakes during development. A relational database supports linked records and transactions. A server-side validation layer is essential regardless of the selected framework because the browser must not decide permissions or gate-pass validity.

4.6.3 Development Environment

The team should record the operating system, runtime versions, package manager, database version, browser versions, and required environment variables. The report should be updated with the exact versions actually used in the implementation; suggested tools are not evidence that they are already installed.

- Store configuration in environment variables.
- Never commit passwords, service keys, or private credentials.
- Use separate development and production configurations.
- Keep database migrations and setup instructions under version control.
- Document how to create the first administrator securely.

4.7 Student Module

The student dashboard should display the current request state, recent requests, and actions available for each request. A new request begins with profile details loaded from the

database. The student cannot overwrite the register number, hostel, or other controlled profile fields from the request form.

4.7.1 Request History

The history view should show the request ID, reason, exit time, expected return time, approval state, cancellation state, and movement state. A QR is shown only when the required approvals are complete and the pass remains valid.

- Create a new request.
- View parent and warden decision status.
- View QR for an approved request.
- Cancel a request when policy permits.
- Review actual OUT and IN times when allowed.
- Receive a clear explanation when an action is unavailable.

4.8 Parent Module

The parent dashboard shows requests only for students linked to the authenticated parent account. Each request should display the student identity needed for recognition, reason, planned departure and return, destination if provided, and current approval status.

4.8.1 Parent Approval Policy

The system should support the policy selected by the institution. Under an any-one-parent policy, one linked parent approval completes the parent stage. Under an all-required-parents policy, every parent marked as required must approve. The selected policy must be clear and should not be inferred from how many parent accounts happen to exist.

- Approve or reject a linked student’s pending request.
- View prior decisions and timestamps.
- Receive exit and return notices.
- Receive an overdue notice when the return deadline passes without an IN event.
- Receive a separate late-entry notice with actual return time and lateness duration.

4.9 Warden Module

The warden dashboard displays requests for hostels to which the warden is assigned. A warden cannot decide requests belonging to an unrelated hostel merely by changing a URL or submitting a different request ID.

4.9.1 First-Response-Wins

Any one authorised warden in the assigned list can approve or reject the request. The server checks the request state and warden assignment within a transaction. If two wardens act concurrently, only the first accepted decision is stored as authoritative; the other receives a status message that the request has already been decided.

- Review parent-stage completion.
- Review the submitted outing details.
- Approve or reject with a recorded timestamp.
- View request history for assigned hostels.
- Use audit information to investigate disputed decisions.

4.10 Watchman Module

The watchman interface should be optimised for quick use at the gate. The scan result must be prominent and unambiguous. The application must validate the QR on the backend and should not trust a QR image merely because it can be decoded.

4.10.1 OUT and IN

An OUT scan is permitted only for a valid approved pass within its allowed exit conditions. A return scan records IN after a valid OUT event. The system should preserve the student's ability to return even if the return is late; the lateness should be recorded and notified rather than treated as a reason to block entry.

- Scan the temporary QR code.
- Display ALLOW or DON'T ALLOW with a reason.
- Record operator and timestamp.
- Reject duplicate or out-of-order state changes.
- Accept a valid return and flag late entry when applicable.

4.11 Admin Module

The admin module maintains the master data needed for reliable authorisation. Student profiles should contain the approved identity and hostel information. Parent accounts must be linked explicitly to the correct student. Warden-to-hostel assignments must be maintained so the approval queue can be filtered securely.

4.11.1 Administrative Controls

Administrative changes should be auditable. Disabling an account should prevent new privileged actions while preserving historical records. Changes to a student's hostel should trigger review of current assignment and pending requests according to institutional policy.

- Create, edit, deactivate, and search accounts.
- Maintain student, parent, staff, and hostel records.
- Link multiple parents to a student.
- Assign authorised wardens to hostels.
- Configure parent approval and attachment policy.
- View audit history and export authorised reports.

4.12 Form Design

The request form separates system-managed identity fields from information supplied for a particular outing. System-managed fields are displayed as read-only. Required fields are validated before submission, and optional fields are clearly labelled.

Table 4.4 Request form fields

Field	Source	Rule
Student name	Student profile	Read-only
College email	Student profile	Read-only
Phone number	Student profile	Read-only
Department	Student profile	Read-only
Hostel name	Student profile	Read-only
Register number	Student profile	Read-only
Room number	Student profile	Read-only when available
Reason	Student input	Required; include Other
Exit date and time	Student input	Required; validate against policy
Expected return date and time	Student input	Required; later than exit
Destination	Student input	Optional or policy-configurable
Emergency contact	Student input	Optional; protect as private data
Attachment	Student upload	Optional by default; configurable requirement

4.13 Approval Rules

Parent approvals must complete before the request enters the warden stage. The parent policy is configurable as any-one-parent or all-required-parents. The warden decision is made by any one authorised warden assigned to the relevant hostel. The server records the accepted decision and prevents a second decision from overwriting it.

4.13.1 Cancellation

Cancellation is permitted while a request is awaiting approval. For an approved pass that has not been used for exit, cancellation may be allowed by policy and must invalidate the QR. After OUT has been recorded, ordinary cancellation is not allowed because the student still needs to return through the same movement record.

Table 4.5 Approval and cancellation rules

Situation	Recommended behaviour
Awaiting parent/warden approval	Allow cancellation; record actor, reason, and time
Approved but no OUT event	Allow only under policy; revoke QR
OUT already recorded	Do not cancel ordinary pass; preserve return workflow
Key details changed while pending	Log changes and reset affected approvals
Late return	Accept valid IN scan; flag and notify lateness
Simultaneous warden decisions	Accept first valid decision only

4.14 Database Design

The data model should separate user authentication from domain-specific student, parent, and staff attributes. Foreign keys link a request to its student and link approvals and scans to the request. Unique constraints should be applied to identifiers that must not be duplicated.

4.14.1 Core Entities

The table below is a conceptual design. Exact column names and types should match the database actually implemented. Passwords should be managed by a trusted authentication service or stored only as secure password hashes if authentication is implemented directly.

Table 4.6 Database entities

Entity	Important attributes
UserAccount	id, email/login, role, active status, created_at
StudentProfile	user_id, register_no, name, phone, department, hostel_id, room_no
ParentLink	parent_user_id, student_id, relationship, required_for_approval
Hostel	id, name, block, active status
WardenAssignment	warden_user_id, hostel_id, active_from, active_to
GatePassRequest	id, student_id, reason, exit_at, expected_return_at, status
ApprovalRecord	id, request_id, stage, approver_id, decision, decided_at
QRPass	id, request_id, token_hash, issued_at, expires_at, revoked_at
GateScanEvent	id, request_id, operator_id, action, result, scanned_at
Notification	id, recipient_id, request_id, event_type, created_at, read_at
AuditEvent	id, actor_id, action, target_type, target_id, occurred_at

4.15 Relationships

One student can have many gate-pass requests. One student can be linked to multiple parents. A warden can be assigned to one or more hostels, and a hostel can have multiple authorised wardens. A request can have several approval records, one or more scan events, notifications, and audit entries.

4.15.1 Integrity Rules

Foreign keys should prevent approvals from referencing missing requests or users. The database should enforce unique request identifiers and prevent duplicate authoritative decisions for the same approval stage. Where the chosen database supports partial unique indexes or equivalent constraints, these can help enforce one accepted warden decision per request.

Table 4.7 Relationships and integrity

Relationship	Cardinality / rule
StudentProfile → GatePassRequest	One-to-many
StudentProfile ↔ Parent account	Many-to-many through ParentLink
Warden account ↔ Hostel	Many-to-many through WardenAssignment
GatePassRequest → ApprovalRecord	One-to-many
GatePassRequest → GateScanEvent	One-to-many
GatePassRequest → QRPass	One current valid token; historical tokens may be retained
UserAccount → Notification	One-to-many
UserAccount → AuditEvent	Actor relationship; not every event has a human actor

4.16 Technical Feasibility

The project uses common web application patterns: browser interfaces, a backend service, relational persistence, and QR scanning. Technical feasibility depends on choosing compatible libraries and correctly configuring authentication, storage, hosting, and notification services.

4.16.1 Operational Feasibility

The five role dashboards are intended to align with existing responsibilities. Students submit requests, parents review linked requests, wardens decide assigned requests, watchmen scan at the gate, and administrators maintain data. A short training and a clear fallback procedure are required for real-world use.

4.16.2 Economic and Schedule Feasibility

A student prototype can use free or low-cost development tools, but production hosting, messaging, backups, and support may incur costs. These should be assessed using the actual selected providers rather than assuming all services are permanently free.

Table 4.8 Feasibility assessment

Area	Assessment
Technical	Feasible with standard web and database components; integration must be tested.
Operational	Fits existing roles if institution policies are confirmed.
Economic	Prototype can be low-cost; production operating costs need estimation.
Schedule	Deliver in increments and prioritise the complete core workflow.
Legal/privacy	Obtain institutional approval and follow applicable data policies.

4.17 Testing Strategy

Testing should cover individual functions, interactions between modules, role boundaries, database integrity, scan state changes, notifications, and end-to-end scenarios. Test data should be synthetic or approved for testing. Production student information should not be used casually in development environments.

4.17.1 Test Levels

Unit tests verify validators and policy functions. Integration tests verify database updates and notification creation. Role tests attempt both permitted and prohibited actions. End-to-end tests follow a request from submission through return. Usability tests check whether stakeholders understand status and scan results.

- Test valid and invalid date-time combinations.
- Test parent approval policies with one and multiple linked parents.
- Test two wardens acting concurrently.
- Test QR before approval, after cancellation, and after expiry.
- Test OUT followed by IN, duplicate OUT, duplicate IN, and IN without OUT.
- Test overdue and late-entry notification timing.
- Test file upload restrictions and private file access.

4.18 Test Case Set A

The following test cases are a planned verification matrix. Results should be filled in only after the application is run. A blank or pending result must not be described as a pass.

Table 4.9 Test cases: authentication and requests

ID	Test input / action	Expected result	Status
TC-01	Valid student login	Student dashboard opens	Pending
TC-02	Invalid password	Access denied; no session created	Pending
TC-03	Parent opens unrelated student request	Access denied	Pending
TC-04	Student submits complete request	Unique request saved	Pending
TC-05	Missing reason	Field validation shown	Pending
TC-06	Return time earlier than exit	Submission rejected	Pending
TC-07	Overnight valid request	Accepted when policy permits	Pending
TC-08	Student edits read-only register number	Value cannot be changed	Pending
TC-09	Required attachment missing when policy requires it	Submission rejected	Pending
TC-10	Attachment has disallowed type	Upload rejected	Pending

4.19 Test Case Set B

Approval tests verify the parent-first order and warden assignment. QR tests verify that the token is created only after required approvals and that the backend rechecks validity on every scan.

Table 4.10 Test cases: approval and QR

ID	Test input / action	Expected result	Status
TC-11	Parent approval pending	Warden stage remains unavailable	Pending
TC-12	Required parent rejects	Request becomes rejected	Pending
TC-13	All required parent approvals complete	Warden queue opens	Pending
TC-14	Unassigned warden attempts decision	Action denied	Pending
TC-15	Two assigned wardens decide concurrently	Only first accepted decision persists	Pending
TC-16	Scan before approval	DON'T ALLOW	Pending
TC-17	Scan cancelled QR	DON'T ALLOW	Pending
TC-18	Scan expired QR	DON'T ALLOW	Pending
TC-19	Valid approved exit scan	OUT event recorded once	Pending
TC-20	Duplicate OUT scan	No second OUT transition	Pending

4.20 Test Case Set C

Return tests focus on accurately recording arrival and preserving the ability to return. Notification tests check both creation of an in-app record and delivery through any configured external channel.

Table 4.11 Test cases: return and notifications

ID	Test input / action	Expected result	Status
TC-21	IN scan after valid OUT	IN event recorded	Pending
TC-22	IN scan without OUT	Rejected or flagged by defined policy	Pending
TC-23	Return before deadline	Return notification created	Pending
TC-24	Deadline passes with no IN	Overdue notice created	Pending
TC-25	Late IN scan	IN accepted; separate late-entry notice	Pending
TC-26	Cancellation while pending	Request cancelled; audit entry saved	Pending
TC-27	Cancel after OUT	Ordinary cancellation blocked	Pending
TC-28	Parent notification opened	Read state updated	Pending
TC-29	Notification provider unavailable	Event remains recorded for retry/inspection	Pending
TC-30	Audit record viewed by unauthorised user	Access denied	Pending

4.21 Expected Results

The expected outcome is a working prototype that follows the defined approval and gate-scan lifecycle. Each role should see relevant information and be prevented from performing actions outside its authority. Requests, approvals, QR events, and scans should remain linked through unique identifiers.

4.21.1 Evaluation Measures

Evaluation should report actual observations, including successful test cases, average response time under a stated test setup, the number of invalid transitions blocked, and stakeholder feedback. The report should not invent performance percentages or user satisfaction results before measurements are collected.

4.22 Challenges

Likely challenges include maintaining accurate parent links, defining the parent approval policy, managing warden assignments, handling concurrent decisions, ensuring reliable gate connectivity, and configuring external notifications. These risks can be reduced through explicit rules, integration tests, and a documented operational fallback.

- Record test date, build version, browser, and test account.
- Keep failed test evidence and fix notes.
- Separate test observations from expected results.
- Re-test related functions after a change.

CHAPTER 5
CONCLUSION

5.1 Conclusion

REC Hostel – Smart Digital Gate-Pass Management System is proposed as a role-based web application to connect student requests, parent confirmation, warden decisions, QR validation, and gate movement records. The design replaces disconnected steps with a central request lifecycle and explicit status transitions.

5.2 Phase I Status

This Phase I report establishes the problem definition, roles, scope, requirements, workflow rules, conceptual architecture, database entities, and validation plan. The document is a design and planning report; the final status of implementation must be updated based on the code, deployment, and test evidence available at submission.

5.3 Limitations

The design depends on accurate user registration, correct parent-student links, maintained warden assignments, reliable device access, and a functioning backend. Notifications may be delayed by external providers. A QR-based process also requires a practical plan for poor connectivity or device failure.

5.3.1 Risk Management

Risks should be recorded with likelihood, impact, mitigation, and an owner. High-priority controls include server-side permissions, secure QR tokens, transaction-safe state changes, backups, and a gate procedure for service interruption.

Table 5.1 Risks and mitigation

Risk	Mitigation
Incorrect parent link	Admin verification and change audit
Stale warden assignment	Maintain assignments and test access checks
QR screenshot misuse	Short validity window, token revocation, server checks
Network interruption	Document fallback and later reconciliation procedure
Notification delay	Persistent in-app notifications and retry monitoring
Data exposure	Least privilege, private files, secure sessions, audit logs
Concurrent decisions	Transactional first-response-wins validation

5.4 Phase II Objectives

The next phase should convert the design into a tested application. Implementation should proceed in small increments, with database migrations and a working end-to-end path established early.

5.4.1 Development Priorities

Priority one is authentication, role assignment, profiles, and hostel relationships. Priority two is the request and approval workflow. Priority three is QR generation and scanning. Priority four is notifications, cancellation, reporting, and security hardening.

- Build schema and seed safe test data.
- Implement server-side role and record permissions.
- Build request form and approval queues.
- Implement first-response-wins warden decision.
- Implement QR issue, validation, OUT, and IN.
- Add overdue and late-entry event processing.
- Complete test cases and update report with evidence.

5.5 Future Enhancements

After the core workflow is stable, the application may be extended according to institutional priorities. Enhancements should not weaken the current role boundaries or create a dependence on untested external services.

- Dashboard showing currently OUT students based on recorded scan events.
- Approval reminders and escalation for requests waiting beyond a defined period.
- Offline scan support with secure reconciliation, if operationally approved.
- Multilingual interface and accessibility improvements.
- Configurable institutional reports and aggregate usage analytics.
- Email or push integration with delivery monitoring.
- Retention settings and privacy-focused administrative controls.
- Formal usability review with students, parents, wardens, and watchmen.

5.6 Work Schedule

The schedule below is a suggested sequence and should be updated according to the actual semester calendar and progress. Each phase should finish with a reviewable deliverable.

Table 5.2 Phase II work schedule

Week	Activity	Deliverable
1	Confirm policy and final requirements	Approved requirement checklist
2	Database and authentication setup	Schema and role-based login
3	Admin, student, parent, hostel data	Working profile management
4	Request form and parent workflow	Parent approval lifecycle
5	Warden assignment and decision logic	First-response-wins warden stage
6	QR issue and gate scan flow	Validated OUT/IN records
7	Notifications, cancellation, audit	Integrated workflow
8	Testing, fixes, report update	Test evidence and final documentation

REFERENCES

The following are foundational standards and documentation sources relevant to the design. The exact library versions used in the implementation should be added to the final report. The project's own test results and institutional policies should be cited separately when available.

[1] OWASP Foundation, "OWASP Application Security Verification Standard (ASVS)," project documentation.

[2] OWASP Foundation, "OWASP Cheat Sheet Series," guidance on authentication, access control, file upload, and session management.

[3] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, 2023.

[4] PostgreSQL Global Development Group, "PostgreSQL Documentation," official database documentation.

[5] React team, "React Documentation," official documentation for component-based user interfaces.

[6] TypeScript team, "TypeScript Documentation," official language documentation.

[7] IETF, RFC 9110, "HTTP Semantics," 2022.

[8] ISO/IEC 25010, "Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model," applicable edition to be confirmed by the project team.

Official product documentation should be consulted for implementation details, security guidance, API changes, and supported versions. Reference formatting and access dates should be normalised before final submission.

[9] DENSO WAVE, "QR Code Essentials," official QR Code information.

[10] Git documentation, "Git Reference Manual," version-control documentation.

[11] MDN Web Docs, documentation on HTML forms, validation, JavaScript, and web security fundamentals.

[12] NIST, "Digital Identity Guidelines," SP 800-63 series, for general identity and authentication principles; apply only where relevant to the institution's requirements.

[13] OWASP Foundation, "File Upload Cheat Sheet," recommendations for handling user-provided attachments.

[14] OWASP Foundation, "Authorization Cheat Sheet," guidance on validating access for each request.

[15] Official documentation of the selected hosting, authentication, storage, and notification providers. Add exact provider names, versions, and access dates after the implementation stack is finalised.

APPENDIX A – CO AND SDG RELEVANCE

A.1 Purpose

This page provides a project-level relevance record for discussion with the department. The official course outcomes and institution-approved SDG mapping must be inserted by the project team or department; the following is a suggested alignment, not an official approved mapping.

Table A.1 CO and SDG relevance record

Relevant theme	Project connection	Evidence to provide
Quality education and digital skills	Applies software engineering to a campus process	Requirements, code, and test records
Industry, innovation, and infrastructure	Designs a digital workflow and information system	Architecture and working prototype
Peace, justice, and strong institutions	Promotes traceable approvals and auditable gate records	Approval logs and role checks
Responsible use of technology	Limits data access and protects private records	Permission tests and privacy review

APPENDIX B – PROGRAM OUTCOMES

B.1 Suggested Outcome Evidence

The project may contribute evidence for outcomes involving problem analysis, design of computing solutions, modern tool usage, teamwork, communication, and ethical responsibilities. The department's official outcome statements should be copied verbatim into the final submission.

Table B.1 Suggested outcome evidence

Outcome area	Possible project evidence
Problem analysis	Problem statement, stakeholder roles, and requirements
Design and development	Architecture, state transitions, schema, and implemented modules
Tool usage	Version control, development environment, database, testing tools
Teamwork	Task allocation and review history for both team members
Communication	Report, demonstration, and user instructions
Ethics and privacy	Least-privilege design, private attachments, and audit controls

APPENDIX C – PROGRAM SPECIFIC OUTCOMES

C.1 PSO Evidence

Program-specific outcomes are institution-defined. The table is an editable template showing the kind of evidence that may be relevant. It must be reviewed against the official Computer Science and Design PSO wording before submission.

Table C.1 PSO evidence template

PSO theme (verify official wording)	Project evidence
Computing solution design	Requirements, architecture, database model
Application development	Role-based web interface and workflow implementation
Testing and quality	Test cases, test execution record, defect fixes
Data and security	Access-control checks, data validation, audit trail

APPENDIX D – CO–PO–PSO MAPPING TEMPLATE

D.1 Mapping Instructions

The project team should complete this table using the official course outcomes (CO), program outcomes (PO), and program-specific outcomes (PSO) supplied by the department. Mapping strengths should follow the department's approved scale. No official mapping values are assumed in this draft.

Table D.1 CO–PO–PSO mapping template

CO	PO / PSO	Mapping level	Justification / evidence
CO-1	PO-1	To be assigned	Problem analysis and requirement definition
CO-2	PO-2	To be assigned	Workflow and data model design
CO-3	PO-3	To be assigned	Implementation of a role-based application
CO-4	PO-5	To be assigned	Use of development and testing tools
CO-5	PO-9	To be assigned	Two-member teamwork and review
CO-6	PO-10	To be assigned	Written report and demonstration
CO-7	PO-8	To be assigned	Privacy, security, and responsible data handling